Ellipsoid Obstacle Conditioning for Safe Trajectory Diffusion

DeepSet Encoding with Classifier-Free Guidance

Sugheerth Sreedharan

April 14, 2026

Contents

1 Overview **2**

2 Ellipsoid Representation **2**

3 DeepSet Ellipsoid Encoder **2**
3.1 Motivation . . . 2
3.2 Architecture . . . 2
3.3 Parameter Count . . . 3
3.4 Integration into TemporalUnet . . . 3

4 Classifier-Free Guidance **3**
4.1 Training with Conditional Dropout . . . 3
4.2 Guided Inference . . . 4

5 CFG DPM-Solver-1 Update **4**

6 Integration with CBF Safety Correction **4**

7 Full Algorithm **5**

8 Hyperparameters **5**

9 File Summary **6**

1 Overview

This document describes the ellipsoid obstacle conditioning system added to the VE-SDE trajectory diffusion model for the `custom/largemaze_ellipsoids-v1` dataset. The system has three components:

1. **DeepSet Ellipsoid Encoder** — a permutation-invariant neural network that maps a *set* of up to $K = 5$ ellipsoid obstacles to a single scene-level conditioning vector $z \in \mathbb{R}^{256}$.
2. **Classifier-Free Guidance (CFG) Training** — the ellipsoid conditioning is randomly dropped out during training, enabling the model to learn both conditional and unconditional trajectory distributions in a single network.
3. **CFG + CBF Sampling** — at inference, a guided score estimate is formed from two forward passes (conditional and unconditional), then optionally corrected by the trajectory-level Control Barrier Function (CBF) for hard safety guarantees.

2 Ellipsoid Representation

Each axis-aligned superellipsoid obstacle j is parameterised by four scalars:

$$e_j = (c_{xj},\ c_{yj},\ a_j,\ b_j) \in \mathbb{R}^4 \tag{1}$$

where (c_{xj}, c_{yj}) is the centre and (a_j, b_j) are the semi-axes along x and y respectively.

The obstacle set for a given planning problem is a *set* (unordered collection) of up to $K = 5$ ellipsoids:

$$\mathcal{E} = \{e_1, \ldots, e_N\}, \quad N \leq K \tag{2}$$

If $N < K$, the set is padded with zero vectors $e_j = \mathbf{0}$ so that the encoder always receives a fixed-size input $\mathbf{E} \in \mathbb{R}^{K\times 4}$.

The signed pseudo-distance from a 2-D point $w = (x, y)$ to obstacle j (degree-4 superellipsoid) is:

$$d_j(w) = \sqrt[4]{\left(\frac{c_{xj} - x}{a_j}\right)^4 + \left(\frac{c_{yj} - y}{b_j}\right)^4 + \varepsilon} - 1 \tag{3}$$

with $d_j > 0$ indicating a safe distance and $d_j < 0$ indicating penetration. This signed distance is used directly by the CBF (see `formulation.tex`); the encoder operates on the raw parameterisation e_j instead.

3 DeepSet Ellipsoid Encoder

3.1 Motivation

The obstacle set $\mathcal{E}$ is *unordered*: there is no canonical ordering of obstacles, and the trajectory planner should be invariant to permutations. A standard MLP over the concatenated obstacle list would break this symmetry.

We use the **DeepSets** framework [1], which achieves permutation invariance via:

$$f(\mathcal{E}) = \rho\left(\bigoplus_{j=1}^{K} \phi(e_j)\right) \tag{4}$$

where ϕ is an element-wise (equivariant) transformation, $\bigoplus$ is a permutation-invariant aggregation (here: *max-pooling*), and ρ is a final invariant projection. Intermediate equivariant layers allow elements to "interact" via the global context before pooling.

3.2 Architecture

Input. $\mathbf{E} \in \mathbb{R}^{K\times 4}$, where each row is $e_j = (c_{xj}, c_{yj}, a_j, b_j)$ and $K = 5$.

Embedding.

$$\mathbf{H}^{(0)} = \mathrm{ReLU}\big(\mathbf{E}\, W_{\mathrm{emb}}^{\top} + b_{\mathrm{emb}}\big) \in \mathbb{R}^{K\times d}, \quad d = 128 \tag{5}$$

DeepSet Layers (equivariant). Each layer combines a local (element-wise) path and a global (max-pooled) path:

$$\mathbf{H}_j^{(\ell+1)} = \mathrm{ReLU}\left(\lambda^{(\ell)}\left(\mathbf{H}_j^{(\ell)}\right) + \gamma^{(\ell)}\left(\max_{j'} \mathbf{H}_{j'}^{(\ell)}\right)\right) \tag{6}$$

where $\lambda^{(\ell)}, \gamma^{(\ell)} : \mathbb{R}^d \to \mathbb{R}^d$ are linear maps (with independent weights), and the max is taken element-wise over the K set dimension. Two such layers are applied ($\ell = 0, 1$).

Invariant Pooling.

$$\mathbf{s} = \max_{j=1}^{K} \mathbf{H}_j^{(2)} \in \mathbb{R}^d \tag{7}$$

Output Projection (ρ).

$$z = \rho(\mathbf{s}) = W_2\ \mathrm{ReLU}(W_1\, \mathbf{s} + b_1) + b_2 \in \mathbb{R}^{d_z}, \quad d_z = 256 \tag{8}$$

Null conditioning. When $\mathbf{E} = \mathbf{0}$ (all obstacle slots zeroed), the encoder outputs a fixed vector $z_\emptyset = \rho(\mathbf{0})$, which the model learns to associate with the unconditional distribution (see Section 4).

3.3 Parameter Count

Component	Parameters
Embedding W_{emb}	$4 \times 128 + 128 = 640$
DeepSetLayer 1 ($\lambda + \gamma$)	$2 \times (128 \times 128 + 128) = 32{,}896$
DeepSetLayer 2 ($\lambda + \gamma$)	$2 \times (128 \times 128 + 128) = 32{,}896$
ρ: Linear($128 \to 128$)	$128 \times 128 + 128 = 16{,}512$
ρ: Linear($128 \to 256$)	$128 \times 256 + 256 = 33{,}024$
Total	**116,008**

3.4 Integration into TemporalUnet

The existing conditioning vector for the VE-SDE score network is:

$$c = [\tau(\sigma)\ ;\ \mathrm{Linear}([x_{\mathrm{start}}, x_{\mathrm{goal}}])] \in \mathbb{R}^{32+4} \tag{9}$$

where $\tau(\sigma)$ is the sinusoidal sigma encoding and the second term projects the start/goal positions.

With ellipsoid conditioning, the vector is extended to:

$$\boxed{c = [\tau(\sigma)\ ;\ \mathrm{Linear}([x_{\mathrm{start}}, x_{\mathrm{goal}}])\ ;\ \mathrm{EllipsoidFiLMEncoder}(\mathbf{E})] \in \mathbb{R}^{32+4+256=292}} \tag{10}$$

This vector is injected into every `ResidualTemporalBlock` of the U-Net via the conditioning MLP, which applies a learned affine shift in the channel dimension.

4 Classifier-Free Guidance

4.1 Training with Conditional Dropout

During training we want the model ϵ_θ to estimate:

- $\epsilon_\theta(x_\sigma, \sigma, x_s, x_g, \mathbf{E})$ — noise given the full conditioning (trajectory endpoints *and* obstacles)
- $\epsilon_\theta(x_\sigma, \sigma, x_s, x_g, \mathbf{0})$ — noise without obstacle conditioning (unconditional w.r.t. obstacles)

in a single network, by using a shared *null token* $\mathbf{0}$.

Concretely, given a mini-batch $\{(x_0^{(i)}, x_s^{(i)}, x_g^{(i)}, \mathbf{E}^{(i)})\}_{i=1}^{B}$, the training loss is:

$$\mathcal{L} = \mathbb{E}_{i,\,\sigma,\,\varepsilon}\left[\left\|\epsilon_\theta\Big(x_0^{(i)} + \sigma\varepsilon,\ \sigma,\ x_s^{(i)},\ x_g^{(i)},\ \tilde{\mathbf{E}}^{(i)}\Big) - \varepsilon\right\|^2\right] \tag{11}$$

where the dropout-augmented obstacle input is:

$$\tilde{\mathbf{E}}^{(i)} = \begin{cases} \mathbf{0}_{K\times 4} & \text{with probability } p_{\text{uncond}} = 0.1 \\ \mathbf{E}^{(i)} & \text{with probability } 1 - p_{\text{uncond}} \end{cases} \tag{12}$$

The dropout mask is sampled *independently per sample* in the batch, so both conditional and unconditional examples appear in every update.

4.2 Guided Inference

At inference, the guided score estimate is formed by linearly extrapolating from the unconditional to the conditional prediction:

$$\boxed{\hat{\epsilon}(x, \sigma, x_s, x_g, \mathbf{E}; w) = \epsilon_\theta(x, \sigma, x_s, x_g, \mathbf{0}) + w \cdot \Big[\epsilon_\theta(x, \sigma, x_s, x_g, \mathbf{E}) - \epsilon_\theta(x, \sigma, x_s, x_g, \mathbf{0})\Big]} \tag{13}$$

where $w \geq 0$ is the **guidance scale**. Special cases:

- $w = 0$: unconditional sampling (obstacle-unaware)
- $w = 1$: standard conditional sampling
- $w > 1$: guidance pushes the trajectory more strongly toward the obstacle-free region (*over-guidance*)

A default of $w = 3.0$ is used, amplifying the obstacle-avoidance signal while preserving trajectory smoothness.

5 CFG DPM-Solver-1 Update

The base DPM-Solver-1 step for VE-SDE (no safety) replaces the plain ϵ_θ with the guided prediction $\hat{\epsilon}$:

$$x_{t_i} = x_{t_{i-1}} - (\sigma_{t_{i-1}} - \sigma_{t_i})\,\hat{\epsilon}(x_{t_{i-1}}, \sigma_{t_{i-1}}, x_s, x_g, \mathbf{E}; w) \tag{14}$$

Start (x_s) and goal (x_g) waypoints are re-pinned (*inpainting*) after each step.

6 Integration with CBF Safety Correction

The CBF correction from `formulation.tex` is applied *after* the CFG blend, using the guided prediction $\hat{\epsilon}$ in place of ϵ_θ. The full safe CFG step is:

$$x_{t_i} = x_{t_{i-1}} - (\sigma_{t_{i-1}} - \sigma_{t_i})\,\hat{\epsilon}_s + u^*(x_{t_{i-1}},\, t_{i-1}) \tag{15}$$

where $\hat{\epsilon}_s = \hat{\epsilon}(x_{t_{i-1}}, \sigma_{t_{i-1}}, x_s, x_g, \mathbf{E}; w)$ and the CBF control term (VE case) is:

$$u^*(x, t) = \begin{cases} -\dfrac{\min(0,\, \omega)}{\|\nabla h\|_2^2 + \gamma^{-1} h^2}\,\nabla h & \omega < 0 \\ \mathbf{0} & \omega \geq 0 \end{cases} \tag{16}$$

$$\omega = \dot{\sigma}_t\,\nabla h(x)^\top \hat{\epsilon}_s + \alpha_0\, h(x) \tag{17}$$

Separation of concerns: The learned conditioning ($\mathbf{E}$ fed to the encoder) and the CBF geometry (raw obstacle parameters used to evaluate h and ∇h) are kept separate:

- The encoder learns a *soft*, data-driven avoidance prior embedded in the score function.
- The CBF provides a *hard*, geometry-based safety guarantee that is active regardless of how well the model has learned to avoid obstacles.

This dual approach is robust: even if the model's obstacle conditioning is imperfect (e.g., for out-of-distribution obstacle configurations), the CBF still corrects unsafe steps.

7 Full Algorithm

Algorithm 1 CFG DPM-Solver-1 + CBF Sampling

Require: Trained model ϵ_θ, VE schedule $\{\sigma_i\}_{i=0}^{n}$ (decreasing), obstacle set $\mathbf{E} \in \mathbb{R}^{K\times 4}$, raw obstacle geometry $\{e_j\}_{j=1}^{N}$ for CBF, start x_s, goal x_g, guidance scale w, CBF parameters $k_1, k_2, c, \alpha_0, \gamma$

$x \leftarrow \mathcal{N}(0, \sigma_0^2 I)$ ▷ Pure noise prior
for $i = 0, 1, \ldots, n-1$ **do**
  $\epsilon_c \leftarrow \epsilon_\theta(x, \sigma_i, x_s, x_g, \mathbf{E})$ ▷ Conditional
  $\epsilon_u \leftarrow \epsilon_\theta(x, \sigma_i, x_s, x_g, \mathbf{0})$ ▷ Unconditional
  $\hat{\epsilon} \leftarrow \epsilon_u + w(\epsilon_c - \epsilon_u)$ ▷ CFG blend
  $x_{\text{new}} \leftarrow x - (\sigma_i - \sigma_{i+1})\,\hat{\epsilon}$ ▷ Base ODE step
  Compute $h(x), \nabla h(x), \omega$ using $\hat{\epsilon}$ and raw obstacle geometry
  if $\omega < 0$ **then**
    $x_{\text{new}} \leftarrow x_{\text{new}} - \dfrac{\omega}{\|\nabla h\|^2 + \gamma^{-1} h^2}\,\nabla h$ ▷ CBF correction
  end if
  $x_{\text{new}}[\,0\,] \leftarrow x_s;\;\; x_{\text{new}}[\,-1\,] \leftarrow x_g$ ▷ Inpainting
  $x \leftarrow x_{\text{new}}$
end for
return x

Algorithm 2 CFG Training with Obstacle Dropout

Require: Dataset $\mathcal{D} = \{(x_0, x_s, x_g, \mathbf{E})\}$, dropout probability $p_{\text{uncond}} = 0.1$, noise schedule $\{\sigma_i\}$

for each mini-batch **do**
  for each sample $(x_0, x_s, x_g, \mathbf{E})$ in batch **do**
    Sample noise level $\sigma \sim \{\sigma_i\}$ uniformly
    Sample $\varepsilon \sim \mathcal{N}(0, I)$
    $x_\sigma \leftarrow x_0 + \sigma\varepsilon$
    With prob. p_{uncond}: $\tilde{\mathbf{E}} \leftarrow \mathbf{0}$; else $\tilde{\mathbf{E}} \leftarrow \mathbf{E}$
    $\mathcal{L}_i \leftarrow \|\epsilon_\theta(x_\sigma, \sigma, x_s, x_g, \tilde{\mathbf{E}}) - \varepsilon\|^2$
  end for
  Update θ via Adam on $\frac{1}{B}\sum_i \mathcal{L}_i$
end for

8 Hyperparameters

Parameter	Description	Default
K	Fixed obstacle set size	5
d	DeepSet hidden dimension	128
d_z	Ellipsoid encoder output dimension	256
p_{uncond}	CFG training dropout probability	0.1
w	CFG guidance scale at inference	3.0
$\sigma_{\min}$	VE noise schedule minimum	0.01
$\sigma_{\max}$	VE noise schedule maximum	10.0
N	Number of noise levels	1000
T	Trajectory horizon (waypoints)	dataset-dependent
k_1	CBF obstacle softmin temperature	1.0
k_2	CBF waypoint softmin temperature	1.0
c	CBF softplus sharpness	1.0
α_0	CBF class-K function gain	1.0
γ	CBF regularisation	1.0

9 File Summary

All new files are placed under `EllipsoidalCBFSampling/`:

File	Description
`models/score_net_ellipsoids.py`	`TemporalUnet` + `EllipsoidFiLMEncoder`
`models/ve_diffusion_ellipsoids.py`	`VEDiffusion` with CFG dropout
`models/samplers_ellipsoids_cfg.py`	CFG sampler and CFG+CBF sampler
`train_large_ellipsoids_diffuser.ipynb`	Training notebook
`docs/ellipsoid_conditioning.tex`	This document

References

[1] M. Zaheer, S. Kottur, S. Ravanbhakhsh, B. Poczos, R. R. Salakhutdinov, A. J. Smola. *Deep Sets*. NeurIPS 2017.

[2] J. Ho, T. Salimans. *Classifier-Free Diffusion Guidance*. NeurIPS 2022 Workshop on Score-Based Methods.

[3] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, B. Poole. *Score-Based Generative Modeling through Stochastic Differential Equations*. ICLR 2021.